

Lenguaje Ensamblador

12026

Juan Camilo Acevedo Bedoya | Facultad de Ingenierías

¿Qué es el Lenguaje Ensamblador?

Definición

El lenguaje ensamblador es un lenguaje de programación de bajo nivel que utiliza nemotécnicos para representar las instrucciones de máquina. Cada instrucción ensamblador corresponde directamente a una instrucción que el procesador puede ejecutar.

El lenguaje ensamblador (Assembly) es un lenguaje de programación de bajo nivel que permite escribir instrucciones directamente comprensibles por el procesador.

Es un lenguaje intermedio entre el código máquina (binario) y los lenguajes de alto nivel (C, Python, etc.). Cada tipo de procesador tiene su propio conjunto de instrucciones (ISA, Instruction Set Architecture), lo que significa que un código en ensamblador para una arquitectura como x86 no es compatible con ARM o RISC-V.

Importancia del Lenguaje Ensamblador

Aunque es más complejo que los lenguajes de alto nivel, el ensamblador permite un control preciso del hardware, optimización de recursos y un entendimiento profundo de la arquitectura del computador. Es esencial en el desarrollo de sistemas operativos, controladores de dispositivos y software embebido.

- **Máximo control sobre el hardware:** Se pueden manejar registros, memoria y periféricos de manera precisa.
- **Optimización de rendimiento:** Se logran ejecuciones más eficientes que con lenguajes de alto nivel.
- **Base para entender la arquitectura del computador:** Facilita el aprendizaje de cómo un procesador ejecuta instrucciones.
- **Programación en sistemas embebidos:** Es crucial en dispositivos con recursos limitados (microcontroladores).
- **Seguridad y análisis forense:** Se usa en ingeniería inversa y análisis de malware.

Sintaxis del Lenguaje Ensamblador. Componentes Clave

1

Instrucciones

Son los comandos que el procesador debe ejecutar. Cada instrucción tiene un nemotécnico (por ejemplo, MOV, ADD, SUB) que la representa.

2

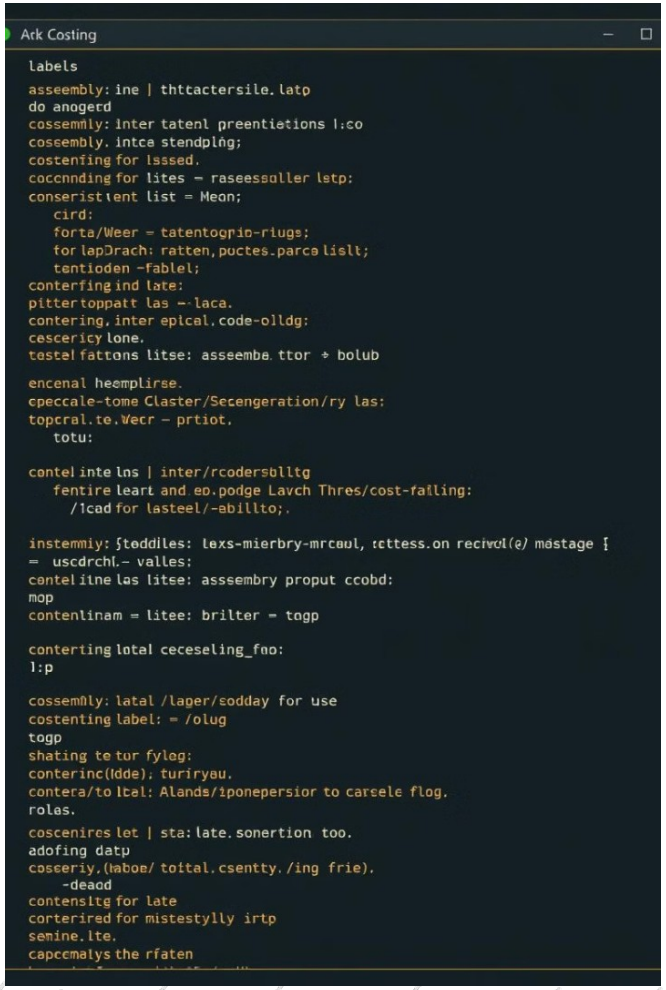
Operadores

Son los datos o direcciones sobre los que operan las instrucciones. Pueden ser registros, variables en memoria o valores inmediatos.

3

Etiquetas

Son nombres simbólicos que representan direcciones de memoria. Facilitan la creación de bucles y el salto entre secciones del código.



Ejemplo Lenguaje Ensamblador x86

```
section .data
    mensaje db "Hola, mundo!", 0

section .text
    global _start

_start:
    mov eax, 4          ; syscall: sys_write
    mov ebx, 1          ; file descriptor: stdout
    mov ecx, mensaje    ; dirección del mensaje
    mov edx, 13         ; longitud del mensaje
    int 0x80            ; llamada al kernel

    mov eax, 1          ; syscall: sys_exit
    xor ebx, ebx        ; código de salida 0
    int 0x80            ; llamada al kernel
```

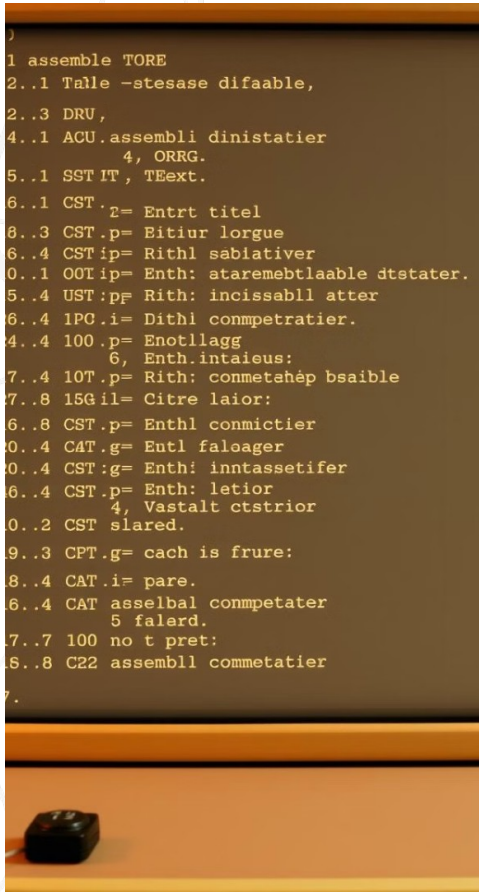
Directivas del Ensamblador

Las directivas de ensamblador son instrucciones especiales que no generan código de máquina, sino que guían al ensamblador en la organización del código, la asignación de memoria y otros aspectos de la traducción del programa.

En otras palabras, las directivas le indican al ensamblador cómo estructurar el código, pero no afectan directamente la ejecución del programa en el procesador.

Las directivas de ensamblador son esenciales para estructurar el código, asignar memoria y optimizar el uso del procesador. No generan instrucciones ejecutables, pero sin ellas, escribir código en ensamblador sería desorganizado y difícil de manejar.

Directivas del Ensamblador
Controlando el Proceso de Ensamblaje



.ORG

Establece la dirección de memoria donde se ubicará el siguiente código o dato.

.EQU

Define una constante simbólica, asignando un nombre a un valor específico.

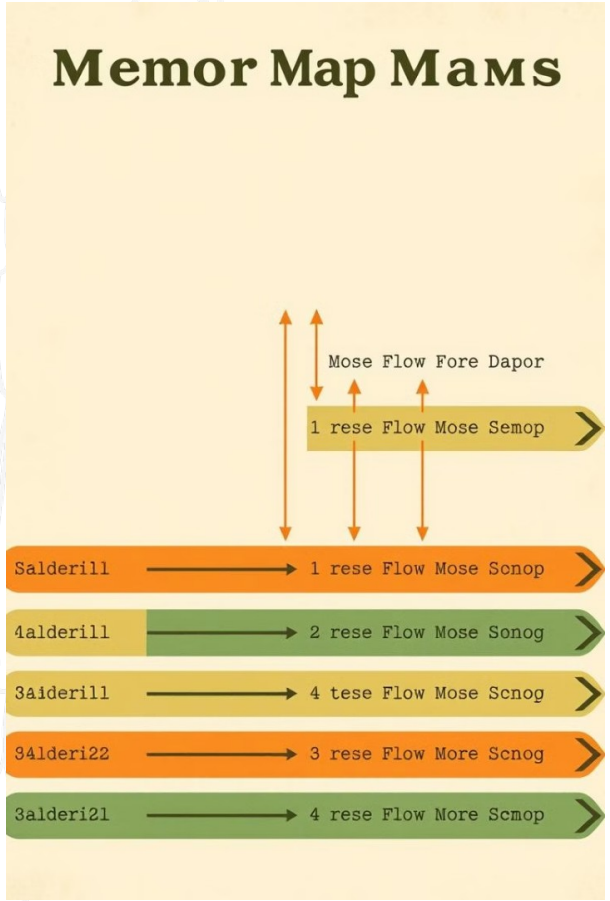
.DATA

Indica el inicio de la sección de datos en el programa.

.TEXT

Indica el inicio de la sección de código ejecutable en el programa.

Modos de Direccionamiento



Los **modos de direccionamiento** en ensamblador definen **cómo** la CPU accede a los datos en memoria o registros para ejecutar una instrucción. Cada microprocesador o microcontrolador tiene un conjunto específico de modos de direccionamiento.

Directo

El operador especifica directamente la dirección de memoria del dato.

Indirecto

El operador contiene la dirección de memoria que contiene la dirección del dato.

Registro

El operador es un registro que contiene el dato.

Control de Flujo.
Bucles y Condicionales en Ensamblador

Saltos (JMP)

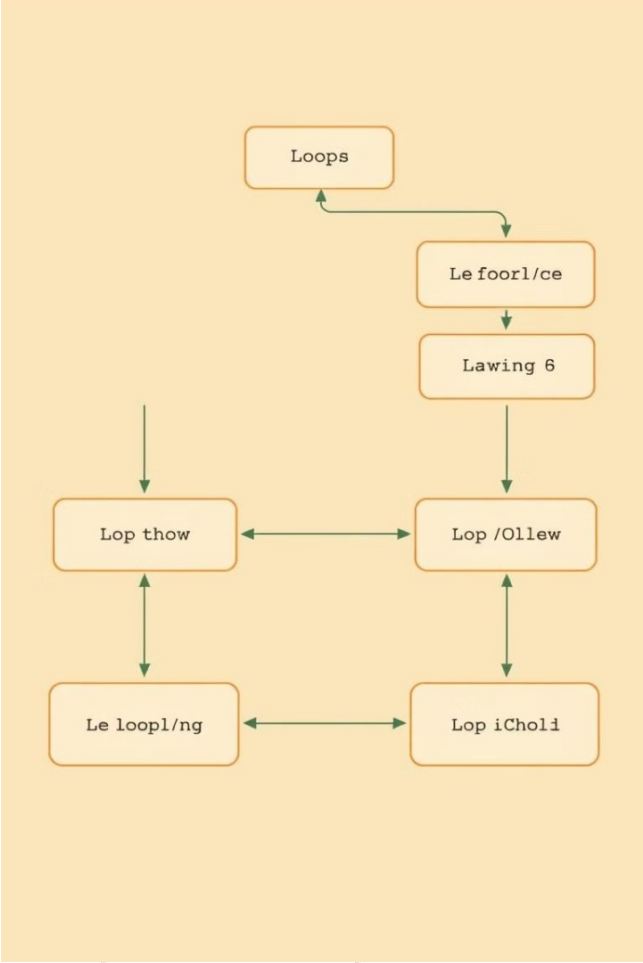
Transfieren el control a una dirección específica del programa.

Saltos Condicionales (JE, JNE, JG, JL)

Transfieren el control si se cumple una condición específica (igual, no igual, mayor, menor).

Bucles (LOOP)

Repiten un bloque de código un número determinado de veces.



Interacción con el Hardware
Puertos y Periféricos

Instrucciones IN y OUT

Se utilizan para leer datos de los puertos de entrada y escribir datos en los puertos de salida.

Puertos de E/S

Permiten la comunicación entre el procesador y los dispositivos periféricos.

Controladores de Interrupciones

Gestionan las solicitudes de interrupción de los periféricos.

Instrucciones Aritméticas y Lógicas Manipulando Datos a Bajo Nivel

1

Aritméticas

ADD (suma), SUB (resta), MUL (multiplicación), DIV (división). Permiten realizar operaciones matemáticas básicas.

2

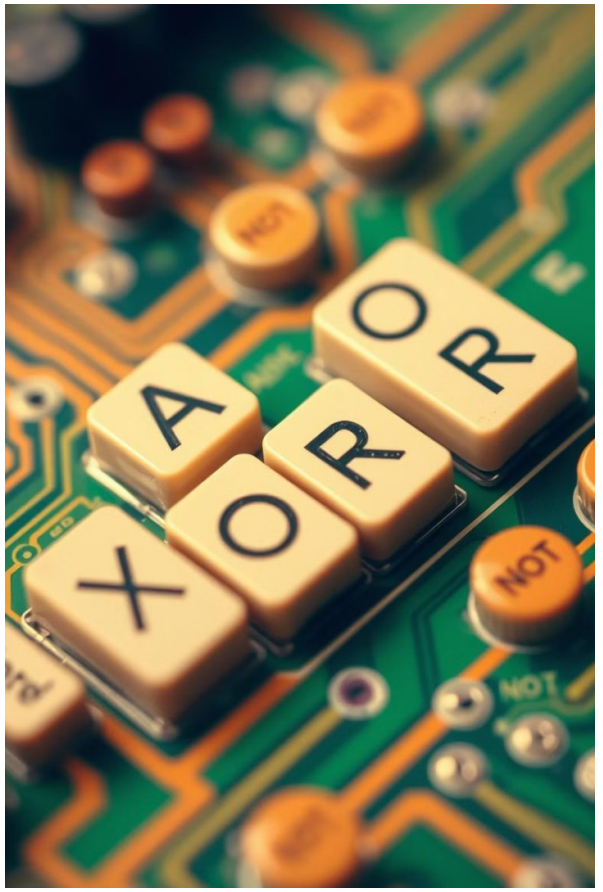
Lógicas

AND, OR, XOR, NOT. Operan sobre los bits individuales de los datos, realizando operaciones lógicas.

3

Desplazamiento

SHL (desplazamiento a la izquierda), SHR (desplazamiento a la derecha). Mueven los bits de un dato hacia la izquierda o la derecha.



MIPS

MIPS significa Microprocessor without Interlocked Pipeline Stages (Microprocesador sin etapas de canalización entrelazadas).

Es una arquitectura de microprocesador que utiliza un enfoque de RISC (Reduced Instruction Set Computing o Computación de Conjunto de Instrucciones Reducido).

Aunque su popularidad en las computadoras de escritorio ha disminuido, la arquitectura MIPS sigue siendo relevante en sistemas integrados como enrutadores, consolas de videojuegos, decodificadores de televisión y dispositivos especializados, debido a su eficiencia y bajo consumo de energía.

Características MIPS

- **Instrucciones simples:** MIPS se basa en un conjunto de instrucciones relativamente pequeño y simple, donde cada instrucción está diseñada para ejecutarse de manera rápida, idealmente en un solo ciclo de reloj.

- **Pipelining (Canalización):** Utiliza una técnica de canalización de 5 etapas (búsqueda, decodificación, ejecución, acceso a memoria y escritura de resultado) para procesar instrucciones de manera más eficiente.

- **Diseño "Load/Store":** La arquitectura solo permite que las operaciones de datos se realicen en los registros del procesador. Para interactuar con la memoria, se usan instrucciones específicas de carga (load) y almacenamiento (store).

Estructura General de MIPS

Procesador RISC

Arquitectura de 32 bits con enfoque en instrucciones simples que se ejecutan en un solo ciclo de reloj, maximizando el rendimiento

Conjunto Reducido

Aproximadamente 180 instrucciones bien definidas que facilitan el aprendizaje y optimizan la velocidad de ejecución

32 Registros

Banco de registros de propósito general que permiten operaciones rápidas sin necesidad de acceder constantemente a memoria

Esta arquitectura simplificada permite que el hardware sea más eficiente y que el compilador pueda optimizar mejor el código, facilitando tanto el aprendizaje como la implementación.

Registros del Procesador MIPS

1. **$zero, $at**

   $zero: Siempre contiene el valor 0, inmutable

   $at: Reservado para el ensamblador (assembler temporary)

2. **$v0-$v1, $a0-$a3**

   $v0-$v1: Valores de retorno de funciones

   $a0-$a3: Argumentos para funciones (primeros 4)

3. **$t0-$t9, $s0-$s7**

   $t0-$t9: Variables temporales (no preservadas en llamadas)

   $s0-$s7: Variables salvadas (preservadas en llamadas)

4. **$k0-$k1, Especiales**

   $k0-$k1: Reservados para el kernel del sistema operativo

   $gp, $sp, $fp, $ra: Puntero global, puntero de pila, frame pointer y dirección de retorno



Convención de Uso de Registros

Registro	Nombre	Uso	Preservación
$at	Assembler Temporary	Reservado para el ensamblador	No
$v0-$v1	Values	Resultados de funciones	No
$a0-$a3	Arguments	Argumentos para funciones	No
$t0-$t9	Temporaries	Variables temporales	No
$s0-$s7	Saved	Variables que perduran	Sí

Seguir estas convenciones es fundamental para la compatibilidad del código y el correcto funcionamiento de las llamadas a subrutinas. Los registros que deben preservarse requieren que se guarden y restauren en la pila al entrar y salir de funciones.

Ejemplos en MIPS

Instrucción	Descripción
add $t0, $t1, $t2	Suma los registros $t1 y $t2, guarda el resultado en $t0.
lw $t0, 0($t1)	Carga una palabra desde la dirección de memoria apuntada por $t1 en $t0.
sw $t0, 0($t1)	Guarda la palabra en $t0 en la dirección de memoria apuntada por $t1.

Estos son algunos ejemplos básicos en MIPS que ilustran la sintaxis y el uso de las instrucciones más comunes. MIPS es una arquitectura RISC ampliamente utilizada en la enseñanza de la arquitectura de computadoras y el lenguaje ensamblador.

Tipos de Instrucciones en MIPS

1

Formato R (Register)

Operaciones entre registros

Ejemplo: add, sub, and, or, slt

Formato: operación $destino, $fuente1, $fuente2

2

Formato I (Immediate)

Operaciones entre registros y un valor inmediato

Ejemplo: addi, lw, sw, beq, bne

Formato: operación $destino, $fuente, inmediato

3

Formato J (Jump)

Saltos a direcciones de memoria

Ejemplo: j, jal

Formato: operación etiqueta

Estos tres formatos constituyen la base del conjunto de instrucciones MIPS, cada uno optimizado para diferentes tipos de operaciones y necesidades de programación.

Instrucciones Tipo R. Estructura y Uso

Características Principales

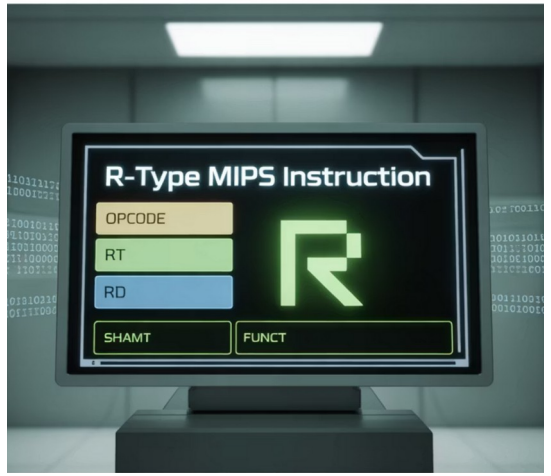
- Operan exclusivamente con registros
- Incluyen operaciones aritméticas y lógicas
- Formato uniforme con tres registros
- Codificación en 32 bits con campos específicos

Instrucciones Comunes

- add, sub: Operaciones aritméticas
- and, or, xor: Operaciones lógicas
- slt: Comparación (set if less than)
- sll, srl: Desplazamientos (shifts)

```
# Ejemplos de instrucciones tipo R

add $s1, $s2, $s3   # $s1 = $s2 + $s3
sub $t1, $t2, $t3   # $t1 = $t2 - $t3
and $s4, $s5, $s6   # $s4 = $s5 & $s6
or  $t4, $t5, $t6   # $t4 = $t5 | $t6
slt $s7, $s0, $t0   # $s7 = ($s0 < $t0) ? 1 : 0
```



Instrucciones Tipo I. Estructura y Aplicación

Características

- Operan entre un registro y un valor inmediato o dirección
- Codificación de 32 bits con campo inmediato de 16 bits
- Incluyen instrucciones de memoria, saltos condicionales y operaciones con constantes

Estas instrucciones son fundamentales para operaciones con constantes, acceso a memoria y control de flujo del programa.

```
# Operaciones con inmediatos

addi $s0, $s0, 10       # $s0 = $s0 + 10
andi $t0, $t1, 0xFF     # $t0 = $t1 & 0xFF
lw   $t0, 4($s0)        # $t0 = Memoria[$s0+4]
sw   $t1, 8($s0)        # Memoria[$s0+8] = $t1
beq  $t0, $t1, etiq     # Salta si $t0 == $t
bne  $s0, $zero, loop   # Salta si $s0 != 0
```

Instrucciones Tipo J. Saltos y Subrutinas

Características

- Permiten saltos incondicionales a cualquier parte del programa
- Direccionamiento directo a etiquetas
- Codificación de 32 bits con campo de dirección de 26 bits
- Fundamentales para implementar llamadas a funciones y estructuras de control

Instrucciones Principales

- j: Salto incondicional
- jal: Jump and link (salto y enlace para subrutinas)
- jr: Jump register (salto al contenido de un registro)

Manipulación Avanzada de Registros

Movimiento de Datos

- move $destino, $origen
- li $destino, valor
- la $destino, etiqueta

Permiten copiar datos entre registros o cargar valores/direcciones inmediatas.

Operaciones con Memoria

- lw $t0, offset($s1)
- sw $t0, offset($s1)
- lb/sb (byte), lh/sh (halfword)

Facilitan la carga / almacenamiento desde / hacia la memoria principal.

Operaciones Especiales

- mfhi/mflo: Mover desde HI/LO
- mthi/mtlo: Mover hacia HI/LO
- mult/div: Multiplicación/división

Interactúan con registros especiales para operaciones aritméticas complejas.

Ejemplo. Bucle WHILE-DO

Pseudocódigo:

```
i = 0
while (i < 10)
do {
        // cuerpo del bucle
        i = i + 1
}
```

Este bucle ejecuta su cuerpo mientras la variable i sea menor que 10, incrementando i en cada iteración.

Implementación en MIPS:

```
 # Inicializar i = 0

     li $s3, 0         # $s3 = i = 0
     li $s4, 10        # $s4 = 10

Loop: # Verificar condición i < 10

     slt $t0, $s3, $s4          # $t0 = (i < 10)
     beq $t0, $zero, End        # Si falso, salir

# Cuerpo del bucle
     # [Aquí van las instrucciones]

# Incrementar i
     addi $s3, $s3, 1  # i = i + 1

# Volver al inicio del bucle
     j Loop

End: # Continuar con el programa
```

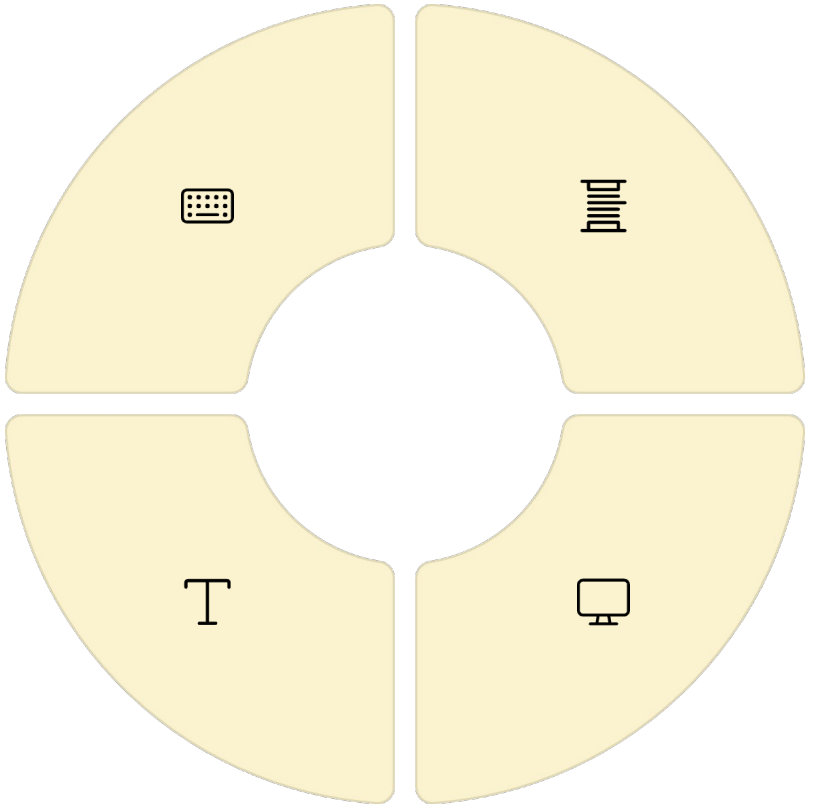
Operaciones de Entrada / Salida (E/S)

Lectura de Enteros

```
# Leer entero del teclado
li $v0, 5
Syscall
# Resultado en $v0
```

Lectura de Cadenas

```
# Leer string
li $v0, 8
la $a0, buffer
li $a1, longitud
syscall
```



Impresión de Cadenas

```
# Imprimir string
li $v0, 4
la $a0, mensaje
syscall
```

Impresión de Enteros

```
# Imprimir entero
li $v0, 1
move $a0, $t0
syscall
```

MIPS utiliza llamadas al sistema (syscalls) para operaciones de E/S, configurando $v0 con el código de operación y usando los registros $a0-$a3 para parámetros. Este mecanismo estandarizado permite interactuar con el sistema operativo de manera consistente.

A la Verdad, por la fe y la ciencia

UCO
Universidad Católica de Oriente
Vigilada Mineducación

Renovación Acreditación Institucional en ALTA CALIDAD



¡Gracias!